

IT7075 · APPLIED AI FOR CYBERSECURITY · MODULE 1

The CyberAI Landscape

Three families of AI, where they help a defender, and why every AI system you build is itself something an attacker will try to break.

Chengcheng Li, PhD · School of Information Technology · University of Cincinnati

Five parts · one live demo

PART 1

Why AI for Security — and What AI Is

Why this is suddenly practical, and the vocabulary you need before we touch any code.

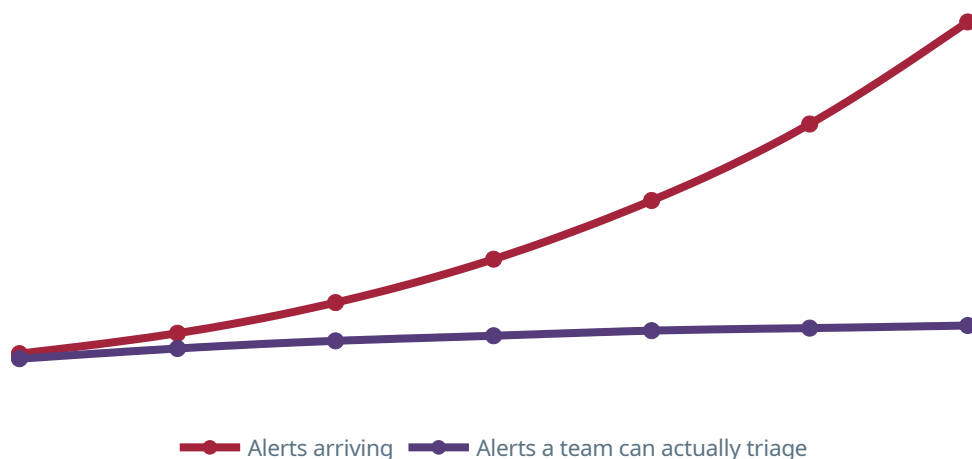
MODULE MAP

- **Why AI for Security**
- AI in the SOC: Tool & Target
- Your First AI Workflow
- A Little Python
- Frameworks, Ethics & Your Week

Why this course, why now

The volume of alerts has outgrown the number of people available to read them.

The triage gap (schematic — shape, not measured data)



Machine-speed attacks

Automation lets one adversary run campaigns that used to need a team.

Alert fatigue

When everything is urgent, nothing is. Real signals get closed unread.

AI is now practical

Triage, detection and enrichment that were research demos in 2020 now ship.

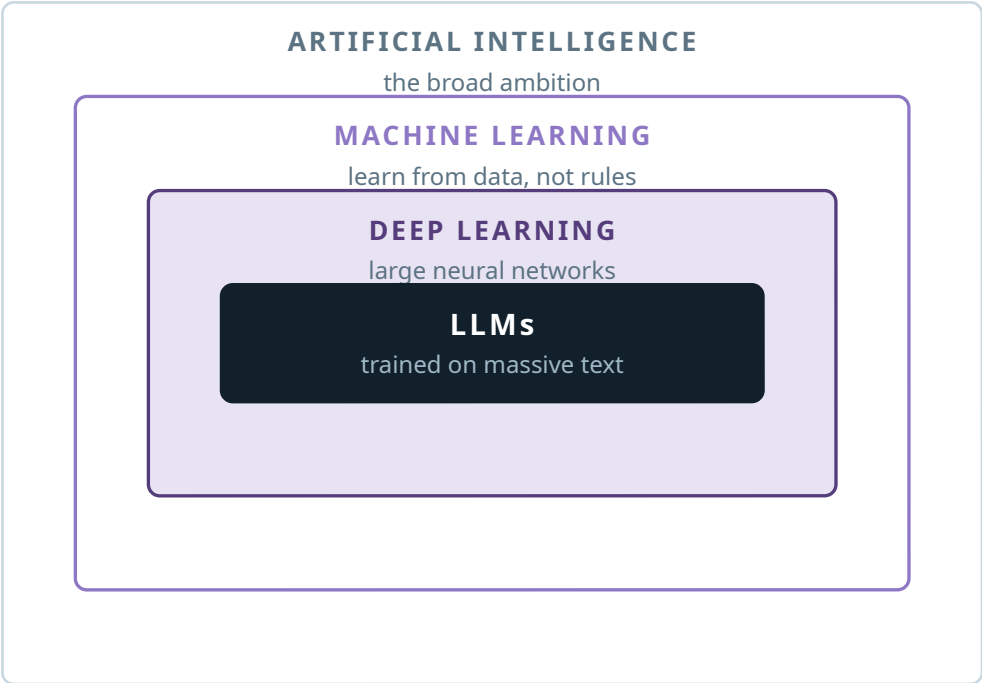
AI is also a target

Models can be fooled, hijacked and poisoned — a new surface to defend.

The claim this course makes: AI does not replace the analyst — it changes what the analyst spends attention on. The gap in that chart never closes by hiring. It closes by putting a first-pass reader in front of the queue and keeping a human on the decisions that matter. By Week 16 you will have built several of those readers yourself.

AI, machine learning, deep learning, LLMs

Four words that nest inside each other — and get used as if they were synonyms.



Term	What it actually means	A security example
AI	Any system doing something we'd call intelligent	The umbrella word in a vendor pitch
Machine learning	Learns patterns from labelled data instead of hand-written rules	Classifying a URL as phishing from its features
Deep learning	ML using large multi-layer neural networks	Detecting malware from raw bytes
LLM	A deep network trained to predict text; reads and writes language	Summarising an incident; triaging an alert

House rule for this course: when I say "AI" without qualifying it, I almost always mean an LLM. When I mean something narrower, I will name it.

Three families of AI you will use

Knowing which to reach for is half the skill. Real systems combine all three.

Classic ML

Patterns in structured data

Phishing-URL classifier · malware detection · anomaly scoring

LLMs

Language and code

Alert triage · incident summaries · code review · log explanation

Agentic AI

Plan and act over steps, using tools

Orchestrated enrichment · multi-step investigation · automated response

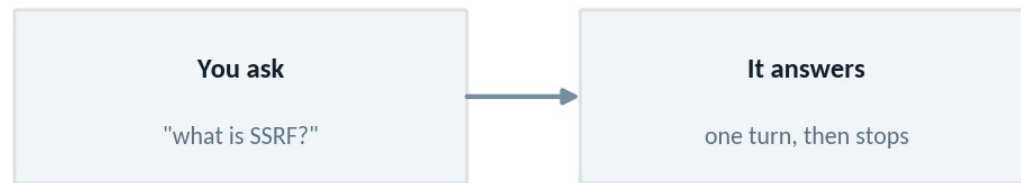
Family	What it eats	What it gives back	Cost to run	Explains itself?
Classic ML	Rows of numbered features	A label and a confidence score	Cents; runs on a laptop	Feature weights — inspectable
LLM	Free text, code, logs	Text: a verdict plus reasoning	Per token, or a local GPU	Writes an explanation you must verify
Agentic AI	A goal, plus tools it may call	A sequence of actions and a result	Many model calls per task	A trace of steps you can audit

Watch the last column. Each family explains itself in a different currency — weights, prose, or a trace. Knowing which kind of evidence you will be able to show an auditor should shape the choice before you write any code.

From assistant to agent

The shift this course is about, in one picture: from answering a question to pursuing a goal.

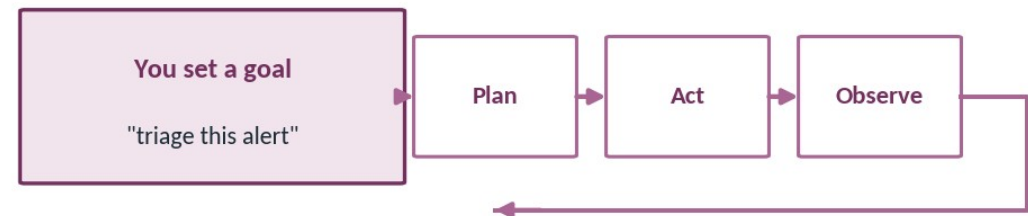
AI ASSISTANT • passive, command-and-response



*you supply the plan, the steps,
and the judgement of when it is done*

"Their defining characteristic is passivity."

AI AGENT • proactive, goal-directed



repeat until the goal is met

*it decomposes the task, chooses tools,
and judges its own progress*

*"proactive, goal-oriented digital entities that can
reason, plan, and execute complex tasks"*

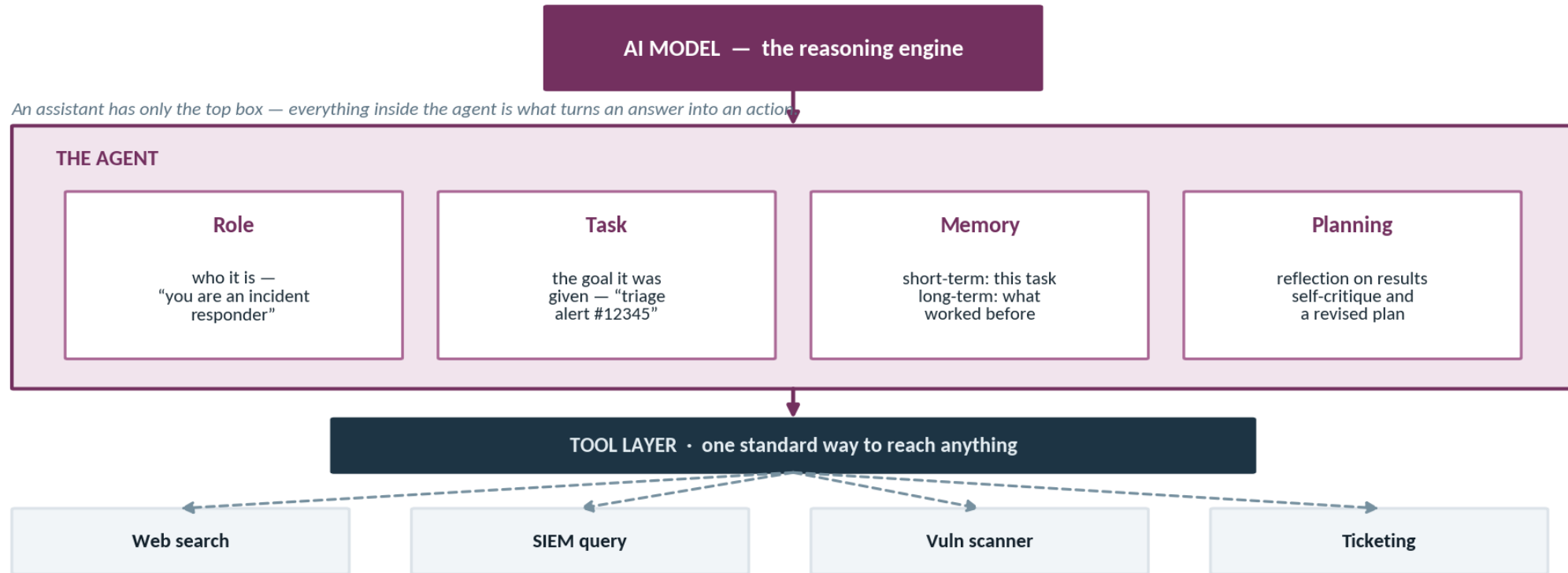
The distinction that organises the whole course. Capability and risk arrive together: the moment a system chooses its own next action, "was that authorised?" becomes a real question rather than a hypothetical one.

Where we are going

Modules 1–7 build the parts. Module 8 assembles them into something that acts on its own.

What is inside an agent

Role, task, memory and planning — sitting on top of a model, and on top of a layer of tools.



Each box becomes a module. Memory and retrieval in module 4, orchestration in 5 and 6, the tool layer in 7, the whole loop in 8. Structure adapted from Santos, Agentic AI for Cybersecurity, Ch. 1, Fig. 1-4.

Four kinds of agent

The reasoning pattern is the design decision. Match it to the task rather than reaching for the most capable one.

	Routing	Query planning	ReAct	Dynamic planning
Primary goal	Find the single best source, model or tool for a query.	Break a complex query into parts and synthesise a complete result.	Accomplish a goal by selecting and using a sequence of tools.	Form a high-level strategy, execute it, and adjust as new information arrives.
Core mechanism	Classification and selection from a predefined list.	Decomposition and synthesis — route sub-questions, combine answers.	A reason–act loop: reason, act with a tool, observe, repeat.	Plan–execute: a planner LLM writes the plan, an executor runs each step.
When to use it	Several distinct, well-defined sources and the problem is choosing one.	Questions no single source can answer.	The steps are not known in advance.	Long-running tasks where the initial conditions change.

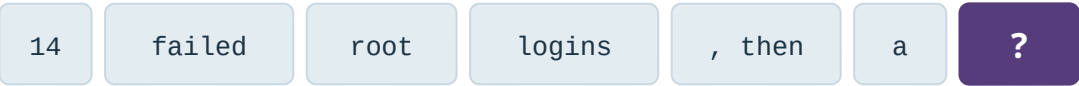
Reach for the simplest pattern that fits. The instinct is that dynamic planning is “best”. It is the most expensive and the hardest to debug — and a great deal of real security work is a routing agent.

Adapted from Santos, Agentic AI for Cybersecurity, Table 1-1.

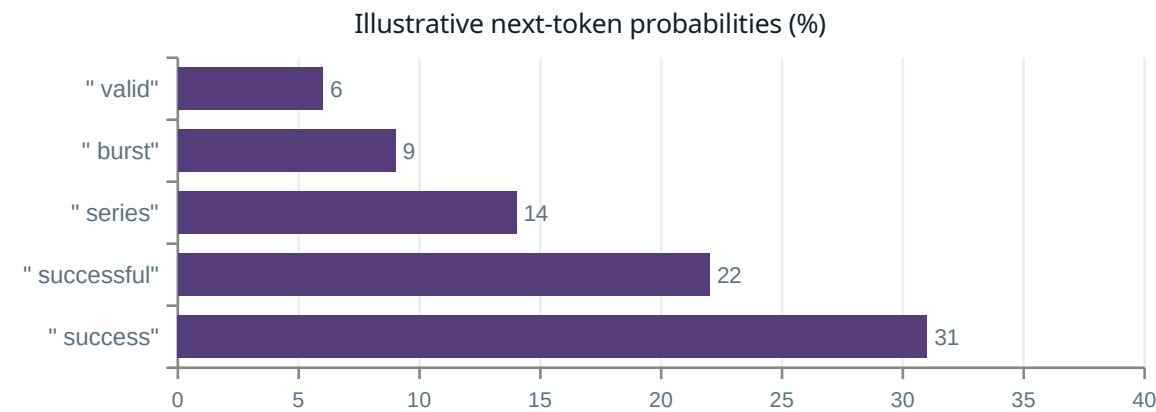
What an LLM actually does

One mechanism — predict the next token — repeated until it has written a response.

THE MECHANISM



your prompt, split into tokens



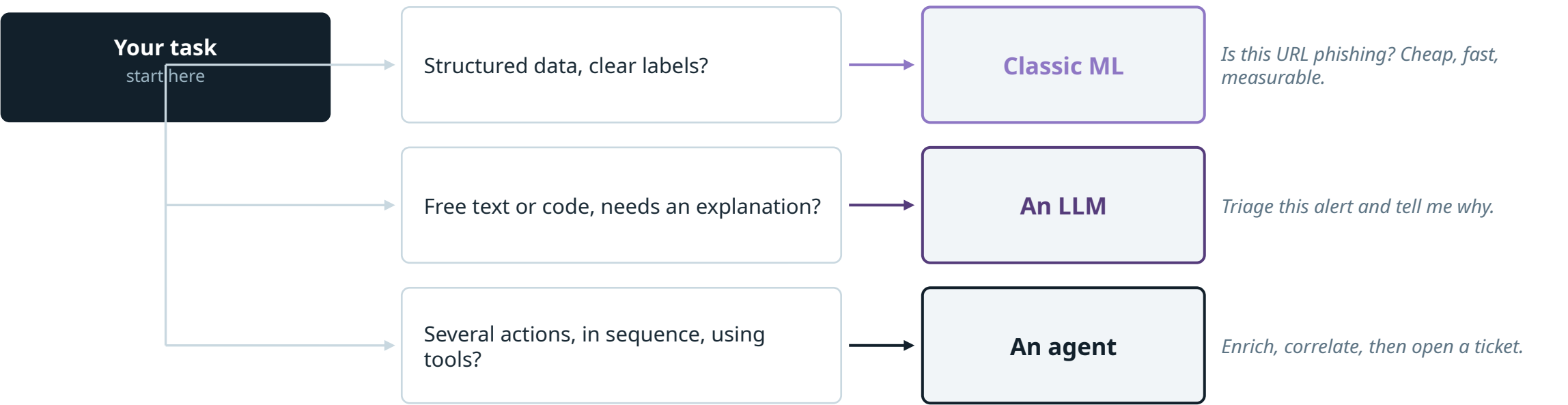
Then it does it again. It picks one, appends it, and re-runs the whole thing. Fluency is the by-product.

Term	What it means for you
Token	Roughly a word-piece. The unit APIs bill in — cost is token count.
Context window	How much text it can consider at once. Its only short-term memory.
System message	A first instruction that sets the role. Not privileged — just early.
Temperature	How much randomness in the pick. Zero for reproducible security work.

The consequence you must internalise: nothing in this mechanism consults a database, and nothing checks whether the output is true. The model produces text that is statistically plausible given what came before. A fabricated CVE number is not a malfunction — it is the same machinery working normally on a prompt where plausible and correct happened to diverge.

Which one do I reach for?

A rule of thumb that keeps you out of most trouble.



Tempting mistake	Why it hurts	What to do instead
An LLM for tabular classification	Slower, costlier, and less accurate than a model you can train in a minute	Train a small classifier; use the LLM to explain its output
One giant prompt for a multi-step job	No trace, no retry, no way to see which step failed	Break it into steps — that is what an agent is

PART 2

AI in the SOC — Tool and Target

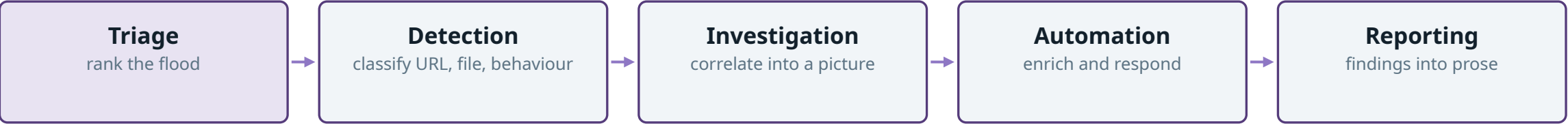
Where AI genuinely helps a defender, where it fails, and why the systems you build are themselves something to defend.

MODULE MAP

-  Why AI for Security
-  **AI in the SOC: Tool & Target**
-  Your First AI Workflow
-  A Little Python
-  Frameworks, Ethics & Your Week

Where AI helps in the SOC

Five places it shows up — and in every one, a human keeps the decision.



Stage	What the AI does	What it saves	What still needs a human
Triage	Reads the queue and ranks by likely severity	The first pass over hundreds of alerts	The escalation call on anything ranked high
Detection	Classifies a URL, file or behaviour as malicious	Manual feature inspection	Tuning the threshold; owning the false negatives
Investigation	Correlates events across sources into a narrative	Pivoting between six consoles by hand	Deciding which thread is worth pulling
Automation	Runs enrichment and drafts a response action	Repetitive lookups and copy-paste	Approving anything irreversible
Reporting	Turns messy findings into a readable writeup	The blank page	Verifying every claim before it ships

Read the last column as the syllabus. Everything in it is judgement under uncertainty — which is precisely what does not automate.

Where AI falls short — so you verify

Four failure modes. All four follow from how the model works, not from it being immature.

Hallucination

Invents CVEs, commands and file paths that look entirely real.

No ground truth

Fluent and confident is not the same as correct — and it cannot tell.

Prompt injection

Untrusted text it reads can carry instructions that hijack it.

Stale, and blind to you

Training has a cutoff, and it knows nothing about your environment.

```
The format is perfect. That is exactly the problem.

> Which CVE is the Apache Struts RCE from that breach?
"CVE-2017-5638 — the Jakarta Multipart parser."    # true

> And the follow-up advisory for the 2023 variant?
"CVE-2023-50164, disclosed December 2023."
# same tone, same shape. Check it yourself.
```

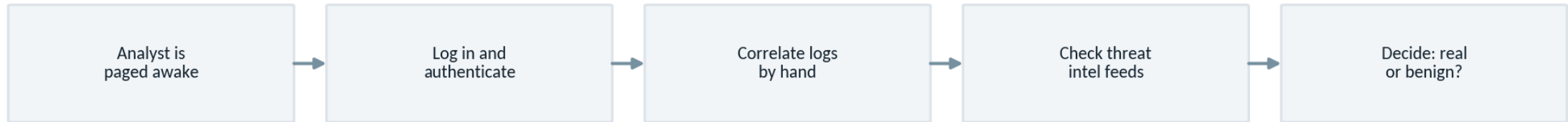
Before you use it	The check
Any CVE or ID	Look it up in the actual database
Any command	Read it. Never paste blind into a shell
Any statistic	Ask where it came from
Any confident tone	Treat as style, not evidence

The rule for the whole course: every model output is a draft to verify, never a fact to trust. This is not pessimism about the technology — it is the correct operating procedure for a system that produces plausible text and has no mechanism for checking truth.

3 a.m., two ways

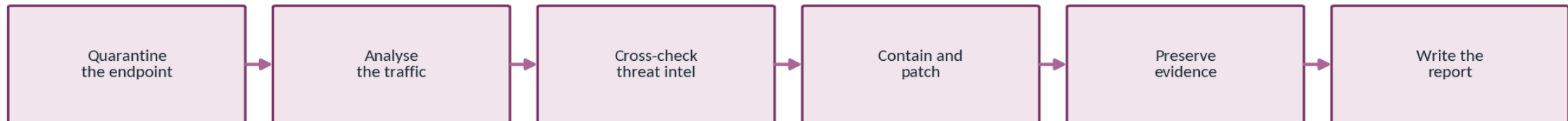
The textbook opens on this scene. It is the clearest picture of what the course is aiming at.

3:00 AM • THE TRADITIONAL PATH



“slow, fraught with the potential for human error, and fundamentally reactive”

3:00 AM • A FLEET OF SPECIALISED AGENTS



Each is a narrow agent with its own tools. A human is still paged — but to review a finished case, not to start one.

Nothing here is a general intelligence. Every one of those is a narrow agent with a role and two or three tools — which is precisely why it is buildable, and what module 8 asks you to build. Scene from Santos, Ch. 1.

AI as a tool — and as a target

A defender who only knows the left-hand column ships attackable systems.

AI FOR DEFENCE

- Triage and rank an alert queue no team can read
- Classify URLs, files and behaviour at machine speed
- Correlate events across sources into a narrative
- Draft the incident report and the remediation steps

AI AS AN ATTACK SURFACE

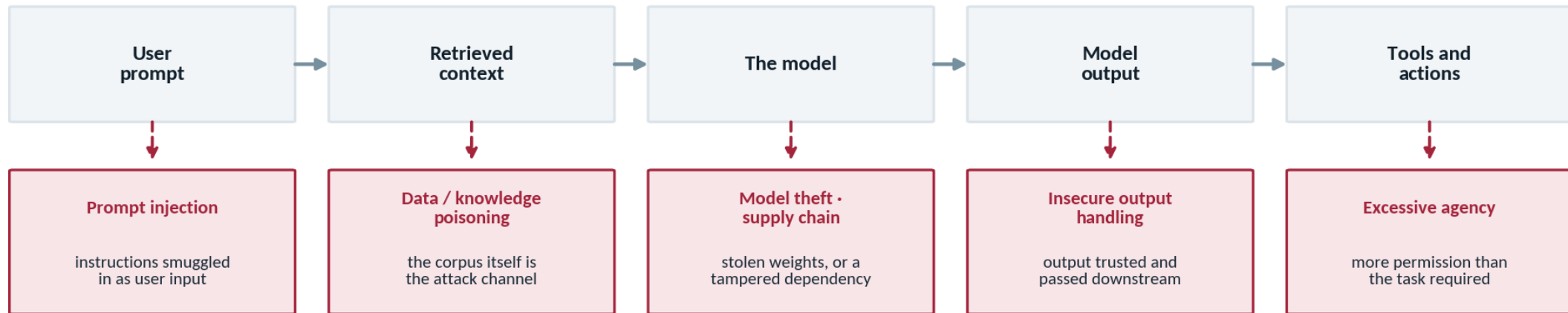
- Adversarial inputs crafted to make a model misclassify
- Prompt injection that hijacks an LLM application
- Data poisoning that corrupts a model during training
- Model and training-data extraction from a deployed system

Reference	What it catalogues	When you reach for it
MITRE ATLAS	Adversarial tactics and techniques against AI systems — the AI cousin of ATT&CK	Threat-modelling anything you build in this course
OWASP Top 10 for LLM Apps	The recurring application-level failures, injection first among them	Reviewing an LLM application before it ships

Where the attacks land

Every stage you added for capability is a stage an attacker can reach. This is the map we return to all term.

Every stage you added for capability is also a stage an attacker can reach



Categories from the OWASP Top 10 for LLM Applications. MITRE ATLAS catalogues the techniques; the NIST AI RMF is how you govern the whole picture.

Read the red row as a syllabus. Prompt injection and data poisoning arrive properly in module 4, insecure output handling in module 9, excessive agency in module 8, and the governance frameworks in module 10. Nothing on this row is hypothetical — every one has a published incident behind it.

What makes a good AI security tool

Five criteria. We return to them every time we evaluate something you built.

1

Correct

Right call on the actual security question

2

Explainable

Reasoning an analyst can check

3

Safe

No data leakage; sensible refusals

4

Bounded

Respects data and network boundaries

5

Fast enough

Fits the analyst's real workflow

Criterion	How you actually test it	What failure looks like
Correct	A held-out set of labelled cases; measure, do not eyeball	Demos beautifully, misses the one alert that mattered
Explainable	Ask a colleague to accept or reject the verdict using only the explanation	A confident verdict with reasoning nobody can check
Safe	Try to make it leak its prompt, its keys, or another tenant's data	Real logs pasted into a third-party API without anyone deciding to
Bounded	Name the worst action it can reach. If it takes a sentence, scope is too wide	A read-only assistant that turned out to be able to write
Fast enough	Time it under a realistic queue, not on one alert	Technically accurate; analysts route around it

PART 3

Your First AI Security Workflow

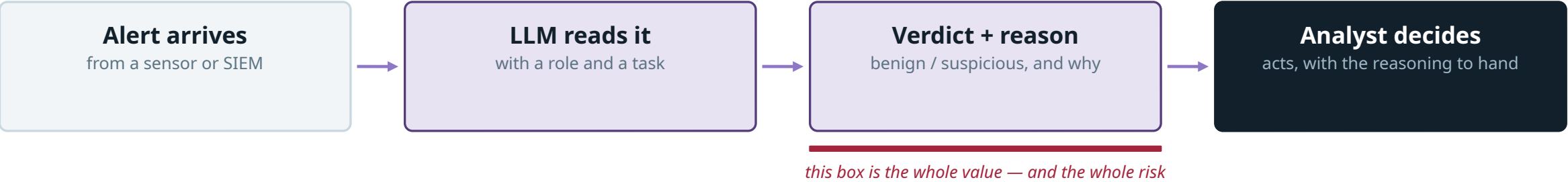
An alert arrives, a model reads it, and an analyst gets a decision with its reasoning attached. Twelve lines of code.

MODULE MAP

- Why AI for Security
- AI in the SOC: Tool & Target
- **Your First AI Workflow**
- A Little Python
- Frameworks, Ethics & Your Week

A motivating example: alert to action

The explanation is what separates a useful assistant from a black box.



Without the explanation	With the explanation	Why it matters here
"Suspicious."	"Suspicious — 14 failed root logins in 40 seconds followed by a success."	The analyst can confirm or reject it in five seconds
You trust it or you redo the work	You audit the reasoning, not just the verdict	A wrong verdict with visible reasoning is caught; a wrong verdict alone is not
Nothing to write in the ticket	The justification is already drafted	The output feeds the next step of the workflow

The demo — the whole thing

Do not worry about the mechanics yet. Notice how little there is.

```
alert_triage.py

import os
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()          # reads OPENAI_API_KEY from a .env file
client = OpenAI()       # the key is never typed into this file

alert = ("14 failed SSH logins for user 'root' from 203.0.113.45 "
        "within 40 seconds, followed by one successful login.")

resp = client.chat.completions.create(
    model="gpt-4.1-mini",
    temperature=0,      # same input, same answer, every run
    messages=[
        {"role": "system",
         "content": "You are a SOC analyst. Classify the alert "
                    "as benign or suspicious, then explain in "
                    "one sentence."},
        {"role": "user", "content": alert}],
    )

print(resp.choices[0].message.content)
```

Line	What it is doing
load_dotenv()	Pulls the key from a file Git ignores
OpenAI()	Finds the key in the environment by itself
system	Sets the role and the output contract
user	The untrusted content — the alert itself
temperature=0	Reproducible: re-runs give the same verdict

Notice what is in the user message. The alert text is attacker-influenceable — it came from a log. Anything written into a filename or a user agent lands in that field. Hold that thought; it is the whole of the prompt-injection module.

The demo, end to end

SimpleAlertClassifier.ipynb in full. Four cells: load the keys, build the prompt, ask each model, compare.

SimpleAlertClassifier.ipynb — CELLS 4, 6 AND 8

```
# --- cell 4: load the keys, safely -----
try:
    from google.colab import userdata          # Colab Secrets
    for k in ("ANTHROPIC_API_KEY", "OPENAI_API_KEY"):
        if userdata.get(k): os.environ[k] = userdata.get(k)
except Exception: pass

try:
    from dotenv import load_dotenv, find_dotenv  # local .env
    load_dotenv(find_dotenv())
except Exception: pass

HAS_ANTHROPIC = bool(os.getenv("ANTHROPIC_API_KEY"))
CLAUDE_MODEL = "claude-haiku-4-5-20251001"
print("Anthropic key:", "found" if HAS_ANTHROPIC else "MISSING")

# --- cell 6: the alert and the prompt -----
alert = "5 failed logins for 'admin' from 203.0.113.7 ..."
PROMPT = f"Classify this security alert as benign or "
        f"suspicious and explain in one sentence: {alert}"

# --- cell 8: ask Claude -----
def classify_with_claude(alert):
    if not HAS_ANTHROPIC:
        return "[skipped — no ANTHROPIC_API_KEY set]"
    msg = Anthropic(api_key=ANTHROPIC_API_KEY).messages.create(
        model=CLAUDE_MODEL, max_tokens=150,
        messages=[{"role": "user", "content": PROMPT}])
    return msg.content[0].text.strip()
```

1 Three key sources, one order

Colab Secrets, then .env, then the shell. Each attempt is wrapped so a missing one is a no-op, not an error.

2 It prints found, never the key

This is what makes the cell safe to project. A key pasted into a cell is a key in your git history.

3 One sentence is the dataset

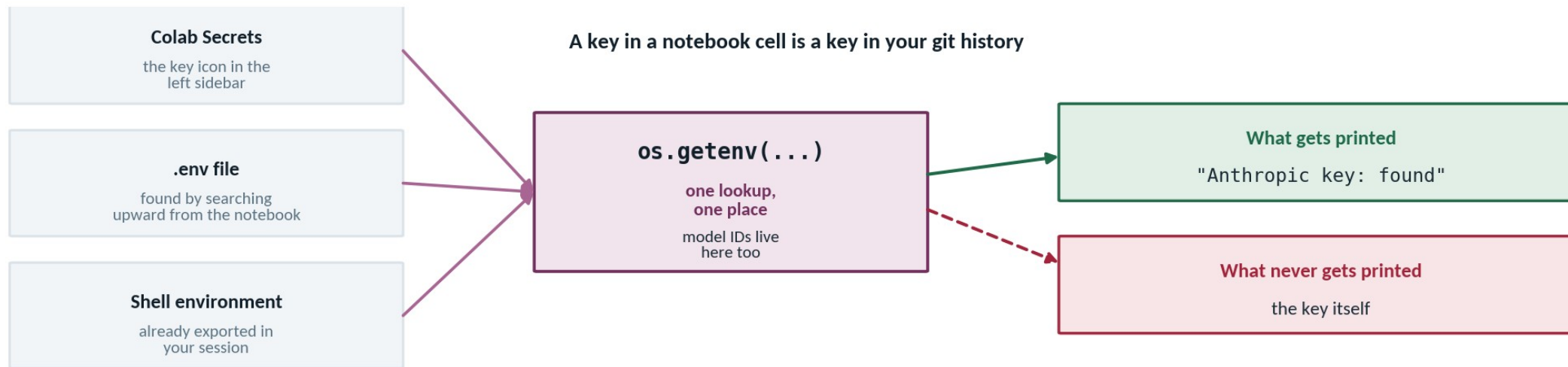
67 characters. Ask the room for a verdict before you run it — get their judgement on the record first.

4 Note the alert parameter

Both classify functions take one. Hold that thought — it comes back two slides from now.

Where the key lives

Three places it may come from, one place the code looks, and one thing that never reaches the screen.



The status line tells you whether a key was found. It can never leak the value — which is what makes it safe to project.

Introduction_to_AI/code/config.py

```
# config.py — the single source of truth
load_dotenv(find_dotenv()) # searches upward
HAS_ANTHROPIC = bool(ANTHROPIC_API_KEY)
CLAUDE_MODEL = "claude-haiku-4-5-20251001"
OPENAI_MODEL = "gpt-5-mini"
```

If a key ever leaks, rotate it

Deleting the message does not help — it is already gone. .env is git-ignored; verify that before your first commit. A leaked key spends real money.

The demo — what comes back

A verdict, and a sentence you can check in five seconds.

Output

Suspicious. Fourteen failed root logins from a single address in 40 seconds followed by a success is a classic brute-force pattern ending in a likely compromise; isolate the host and rotate credentials for that account.

What it got right	What it added	What you must still check
The verdict — this is the correct call	A named pattern: brute force	Is 203.0.113.45 actually external?
The reasoning cites the evidence in the alert	An unrequested remediation step	Did the success come from the same source?
Format followed the instruction	Confidence it has not earned	Is this host in scope for isolation?

Why this example is easy

Any analyst spots this pattern instantly. That is deliberate — the demo shows the shape, not the value.

The value appears at 500 alerts an hour with no clean threshold, where a first-pass reader with a stated reason lets a human spend attention where it counts.

The habit to build now: read the second column and the third together. The model volunteered a remediation nobody asked for. It may well be right — and it also has no idea whether that host can be isolated, or whether it is a domain controller. Helpfulness beyond the question asked is where automation quietly acquires scope.

The argument that does nothing

The notebook's classify functions take an alert and then ignore it. No error, no warning — just the wrong answer.

SimpleAlertClassifier.ipynb · the argument is dead

```
def classify_with_claude(alert):
    ...
    messages=[{"role": "user",
                "content": PROMPT}]
    # ^^^^^ the global
    # ^^^^^ never the argument
```

you call it with a NEW alert:

```
classify("VPN login from a new country")
```

the model actually receives:

```
"...5 failed logins for 'admin' from
203.0.113.7 in 20s, then 1 success"
```

the ORIGINAL alert — silently, with no error

alert_classifier.py · the same demo, done right

```
def classify_with_claude(alert: str) -> str:
    ...
    messages=[{"role": "user",
                "content": PROMPT.format(alert=alert)}]
    # ^^^^^^^^^^^^^
```

you call it with a NEW alert:

```
classify("VPN login from a new country")
```

the model actually receives:

```
"...User connected to the VPN from a
new country at 3am"
```

the alert you asked about

This is the module's thesis, demonstrated on the module's own code. A confident, well-formed answer to a question you did not ask. There is no error and no warning — the only way to catch it is to read the code. Fix: `PROMPT.format(alert=alert)`, exactly as the `.py` script already does.

Three ways the demo ends

Measured, not guessed: no keys, working keys, and the one the delivery guide does not plan for.

NO KEYS AT ALL

the guarded path

```
Anthropic key: MISSING
OpenAI key:    MISSING

[skipped — no ANTHROPIC_API_KEY]
[skipped — no OPENAI_API_KEY]
```

script exits 0 in 0.06 s
notebook runs 7/7 cells, 0 errors

Handled. This is the path the guard was written for.

BOTH KEYS WORKING

the demo as designed

```
Anthropic key: found
OpenAI key:    found

Claude  (claude-haiku-4-5)
OpenAI  (gpt-5-mini)
```

two verdicts to compare

What the delivery guide assumes. Verify it before class.

KEYS SET, API UNREACHABLE

bad key · firewall · campus wi-fi

```
Anthropic key: found
OpenAI key:    found

APIConnectionError: Connection error.
NameError: claude_result is not defined
```

script dies in 2.37 s, exit 1
OpenAI panel never runs

Unhandled. There is no try/except around either call.

THE TWO LINES THE NOTEBOOK IS MISSING

```
try:    result = client.messages.create(...)
except Exception as e: result = f"[error — {type(e).__name__}]"
```

test_api_keys.py already does this

The only file in the module with real exception handling — and the delivery guide never mentions it. Run it before class.

Same prompt, unequal settings

Before you compare two models, check that you gave them the same conditions. This demo does not.

Same alert. Same prompt. Different harness.

CLAUDE

`max_tokens=150`

capped

can be cut off mid-sentence

OPENAI

`no limit set`

runs as long as it likes

"Claude was terser" is then a fact about your code, not about Claude.

CELLS 8 AND 10

```
# Claude
max_tokens=150

# OpenAI — no cap
chat.completions.create(
    model=OPENAI_MODEL,
    messages=[...])
```

And you cannot just add it

The gpt-5 family rejects `max_tokens` on chat completions and wants `max_completion_tokens`. The API surface is not uniform — portable code across providers is harder than it looks.

Before you compare two models, check you gave them the same conditions. Slide 16 asks whether one model is more specific or better justified. As the code stands, a difference in length is a fact about the harness rather than about the models — and gpt-5-mini is a reasoning model, so its latency and cost profile are not comparable to Haiku's in the first place.

Two models, one prompt — and where the key lives

There is no such thing as "the AI says." There is what a particular model said, once.

	Model A	Model B
Verdict	Suspicious	Suspicious
Names the pattern	"classic brute force"	"anomalous authentication sequence"
Volunteers action	Isolate the host; rotate credentials	None — sticks to the question
Hedging	None	"appears consistent with"
Useful to an analyst?	More actionable, more assumed	More cautious, less immediately usable

Same prompt, same alert, different answers. Neither is wrong. They differ in confidence, vocabulary and willingness to volunteer. That variance is a property of the system you are building on, and your design has to survive it.

```
.env — and .gitignore

# .env (never committed)
OPENAI_API_KEY=sk-...

# .gitignore
.env
*.key
```

Keys: the three rules

Never in code, a cell, or a screenshot.
Never in a commit — history keeps it forever, and bots scan public repos within minutes.
If one leaks, rotate it immediately. Assume compromise from the moment of the push.

PART 4

A Little Python

How much programming this course actually needs, why reading matters more than writing, and three free ways to get there.

MODULE MAP

- Why AI for Security
- AI in the SOC: Tool & Target
- Your First AI Workflow
- **A Little Python**
- Frameworks, Ethics & Your Week

You will not write much code — but you must read it

Reading is a much lower bar than writing, and it is the one that actually matters here.

```
If you can follow this, you have enough Python for this course

import pandas as pd

logs = pd.read_csv("auth.log.csv")
failed = logs[logs["result"] == "FAIL"]

by_ip = failed.groupby("src_ip").size()
suspects = by_ip[by_ip > 10]

for ip, count in suspects.items():
    print(f"{ip}: {count} failed attempts")
```

That is brute-force detection. Nine lines, no machine learning. You do not need to have written it — you need to look at it and know that it filters failures, counts them per source address, and keeps the noisy ones. That is the level.

You need to recognise	So that you can
Variables, strings, f-strings	Follow what a cell is building
Lists and dictionaries	Read an API request and its response
Loops and if/else	See where the logic branches
Functions and imports	Know what came from where
A traceback	Fix your own errors instead of stalling

Why bother, if AI can write it?

Because you cannot debug what you cannot read, you cannot adapt an example to your own data, and you cannot tell when generated code is doing something you did not intend. Reading is the review skill.

The Python you actually need

The self-check list is not abstract — every item on it appears in the twenty lines you just read.

alert_classifier.py — AND THE SEVEN THINGS THE SELF-CHECK ASKS ABOUT

```
import os
from config import (ANTHROPIC_API_KEY, HAS_ANTHROPIC,
                    CLAUDE_MODEL, key_status)

PROMPT = ("Classify this security alert as benign or "
          "suspicious and explain in one sentence: {alert}")

def classify_with_claude(alert: str) -> str:
    """Classify the alert; skip cleanly if no key is set."""
    if not HAS_ANTHROPIC:
        return "[skipped — no ANTHROPIC_API_KEY set]"

    from anthropic import Anthropic
    client = Anthropic(api_key=ANTHROPIC_API_KEY)

    msg = client.messages.create(
        model=CLAUDE_MODEL,
        max_tokens=150,
        messages=[{"role": "user",
                  "content": PROMPT.format(alert=alert)}],
    )
    return msg.content[0].text.strip()

print(f"Alert: {alert}")
```

1 Imports · functions · arguments

from-import at the top, and again inside the function — which is why the script starts in 0.06 s even before any network call.

2 if / else, and a default

The guard. One conditional is the whole difference between a clean skip and a traceback in front of the class.

3 Lists and dictionaries

messages=[{...}] — a list containing a dictionary. That exact shape is how every chat API takes its input. Worth ten minutes.

4 Strings, f-strings, printing

The prompt, and the output. Six of the seven checklist items are on this screen; the seventh, loops, is in the compare cell.

Python: a recommended review for Weeks 1–3

Three free options. Pick one that matches how you learn — there is nothing to submit.

RECOMMENDED

LinkedIn Learning

"Python for Non-Programmers"

About two hours, video-led, free to you through UC. The one I recommend most for absolute beginners.

learnpython.org

Interactive, in the browser

Nothing to install. You run each lesson's code in the page. Best if you learn by doing.

A free YouTube course

~2 hours, video

Linked on the module page. Best if you want it playing while you follow along in Colab.

Option	Format	Time	Choose it if
LinkedIn Learning — Python for Non-Programmers	Video, structured	~2 hrs	You have never programmed and want a guided path
learnpython.org	Interactive browser	Self-paced	You would rather type than watch
Free YouTube course	Video	~2 hrs	You want something to follow along with in Colab

Recommended, not assigned. Nothing to submit, no marks attached. Do only as much as you need to read the code card on the previous slide comfortably.

PART 5

Frameworks, Ethics & Your Week

The three references we lean on all term, the ground rules that start today, and what this week asks of you.

MODULE MAP

-  Why AI for Security
-  AI in the SOC: Tool & Target
-  Your First AI Workflow
-  A Little Python
-  **Frameworks, Ethics & Your Week**

Three references we return to all term

Bookmark them today. They come back in almost every module.

MITRE ATLAS

Adversarial threat landscape for AI systems

A catalogue of tactics and techniques used against AI — deliberately structured like ATT&CK, so it reads immediately if you know that.

NIST AI RMF

AI Risk Management Framework

The vocabulary for governing AI risk: govern, map, measure, manage. What you cite when someone asks how the risk was assessed.

OWASP

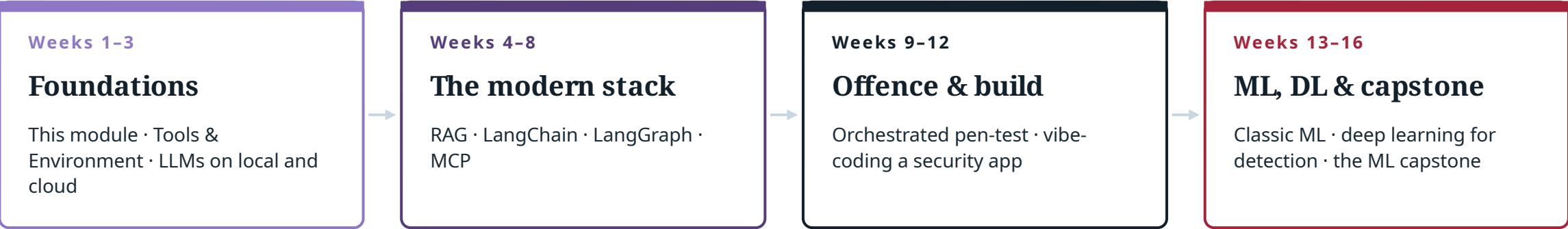
Top 10 for LLM Applications

The recurring application-level failures, with prompt injection at the top. Our checklist before anything ships.

Reference	Answers the question	Where it shows up in this course
MITRE ATLAS	How would someone attack this AI system?	Threat-modelling every mini-project; the adversarial-ML module
NIST AI RMF	How do we govern and document the risk?	The risk section of your capstone report
OWASP LLM Top 10	What goes wrong in LLM applications specifically?	The injection module, and every code review this term

Course roadmap at a glance

Four phases across sixteen weeks — each one builds on the toolchain you set up in the next module.

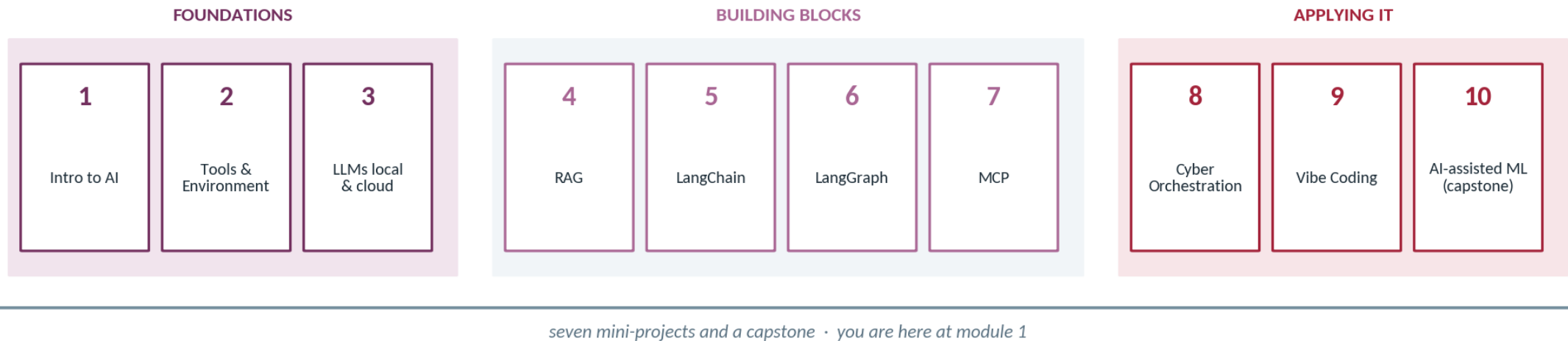


How you are assessed	What it asks of you	What I am looking for
Labs	Stand the thing up and prove it runs	Evidence it worked on your machine, not a copied screenshot
Discussion forum	One insight, in your own words, plus two replies	Your understanding — not a paste from a model
Seven mini-projects	Build something small that answers a security question	Design choices you can defend, and honest limitations
Capstone	An end-to-end ML security project with a written report	The five criteria from Part 2, applied to your own work

The ten-module arc

Where each piece arrives, and why the order is what it is.

You will have built every layer of this stack yourself before you use a framework that hides it



Framework-free first, on purpose. In module 4 you build a retrieval engine by hand from an embedding model and a dot product. In module 5 you do the same thing in fifteen lines of LangChain. Meet the framework first and it is magic — and when it breaks you have nothing to reason with. Write the pieces yourself and the framework becomes a convenience you can see through.

Seven mini-projects and a capstone

They build on each other. The retrieval you write in module 4 becomes the tool your agent calls in module 8, so falling behind compounds rather than staying local.

Ethics and responsible use — from day one

These apply from today through the capstone. Two of them carry legal weight.

Rule	What it means concretely	Why
Work only in isolated labs, or on apps you built	Your Kali VM, the provided targets, your own code. Never a live system you do not own	Unauthorised scanning or access can violate the CFAA and equivalent laws
Use synthetic or provided data	Never real PII, secrets, credentials or live incident data in a prompt	A cloud model call is third-party processing — treat prompts as retained
Keep keys out of code and commits	A .env file that .gitignore excludes; Colab Secrets in notebooks	Public repos are scanned within minutes; history keeps a leaked key forever
Rotate anything that leaks	Immediately, before you finish investigating how	The window between exposure and rotation is the whole risk
Set spend limits on paid APIs	A hard cap in the provider console, today	A runaway loop is a real and common way to lose a term's budget

The two that catch good students: pointing a tool at your employer's network to make a project feel realistic, and pasting a real incident log into a model to get better help. Both are done with good intentions. Both are serious. If you are ever unsure whether something is in scope, ask me first — that costs you a day, and the alternative can cost considerably more.

Your week — and what comes next

One required task, one short self-check, and an optional run of the notebook.

	Do this	What it involves	Required?
1	Watch this lecture and the lab demonstration	The recording and the demo walkthrough	Yes
2	Onboarding assignment on the forum	Introduce yourself, then reply to two classmates	Yes — graded
3	Python reading self-check (1–5)	Five short snippets; say what each one does	Yes — ungraded
4	Start the recommended Python review	Only if the self-check felt uncomfortable	No
5	Run the triage notebook yourself	Then post how it went, and anything that surprised you	Optional

On the self-check: it is ungraded and I never see individual answers — it exists so you can calibrate. If three of the five snippets are opaque to you, that is useful information and the review will fix it in an afternoon. If all five are obvious, skip Part 4 entirely and spend the time on the forum instead.

Next module — Tools & Environment

You stand up the toolchain you will use all semester: editor, Python, notebooks, Git, secrets handling, and the isolated security lab.

Plus, optionally, a free cloud GPU VM through NAIRR.

Do the forum post early. The replies are the point, and they only work if people have posted before the end of the week.

How to succeed in this course

Five habits that separate the students who finish strong.

1 Run everything yourself

Reading a notebook is not the same as running it. The errors are where the learning is.

2 Break it on purpose

Change a parameter, feed it bad input, see what happens. Curiosity beats completion.

3 Verify before you submit

Every CVE, every command, every statistic. An unverified claim costs marks.

4 Ask early

A question in week two is a conversation. The same question in week twelve is a rescue.

5 Write in your own words

The forum and the reports are where understanding gets demonstrated. Pasted text demonstrates nothing.

The one that matters most: number five. Everything else on this slide is technique. That one is the difference between having used AI and having learned something — which is the subject of the next slide.

Use AI as a tutor — the Feynman test

There are no restrictions on AI use in this course. There is one condition.

USING IT AS A VENDING MACHINE

Ask "Write my forum post about RAG."

Get A fluent paragraph.

Post It, unchanged.

Learn Nothing. And your classmates could have generated the same text themselves.

USING IT AS A TUTOR

Ask "My professor asked this and I don't know where to start. Teach me the background with simple examples."

Push "Why does that work? Give me a case where it fails."

Close Shut the tab. Write the post from memory.

Learn If you can explain it in your own words, you understand it.

Why I do not block AI tools: blocking does not work — phones, VPNs, and a world that already runs on this — and more importantly it would be pedagogically backwards, because a model is a genuinely excellent tutor with infinite patience. So use it as much as you like. The condition is the Feynman test: if you can explain it clearly in your own words, you understand it. That is what the forum posts, the mini-project presentations, and the capstone report are actually measuring — not what the AI produced, but what you can explain.

Recap

Five things to carry out of today.

1 Three families, one skill

Classic ML, LLMs, agents. Matching the family to the task is most of the judgement.

2 It predicts plausible text

No database, no notion of truth. Hallucination is the mechanism working normally.

3 Tool and target

Every AI system you build is itself an attack surface. MITRE ATLAS is the catalogue.

4 Read Python, don't write it

Enough to follow a notebook, fix an error, adapt an example. That is the bar.

5 Verify everything

Every output is a draft. Every CVE gets looked up. Every command gets read.

Next: head into the lab and run your first AI security triage workflow yourself. Then Module 2 — Tools & Environment. Welcome aboard.